HISTORY OF JAVA

- Java is a high level, robust, object-oriented and secure programming language.
- Java was developed by ***Sun Microsystems*** (which is now the subsidiary of Oracle) in the year **1995.**
- Before Java, its name was ***Oak*** *(Considered as symbol of strength)*. Since Oak was already a registered company, so James Gosling and his team(***Green team*** **– James Gosling, Mike Sheridan, Patrick Naughton**) changed the name from Oak to **Java**.
- **James Gosling is known as Father of Java.**

FEATURES OF JAVA

- Simple:

Java is very easy to learn, and its syntax is simple, clean and easy to understand.

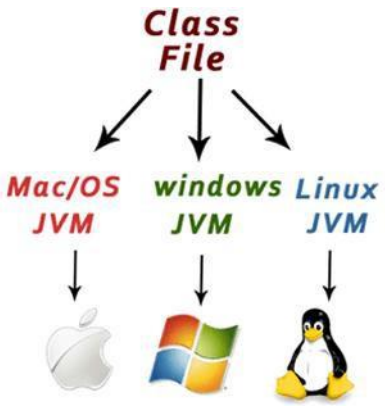
Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.

- Object-oriented:

Java is an object-oriented programming language. Everything in Java is an object.

- Platform Independent:

Java is platform independent. Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.



- Secured:

Java is best known for its security. With Java, we can develop virus-free systems.

- Robust:

It uses strong memory management. Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.

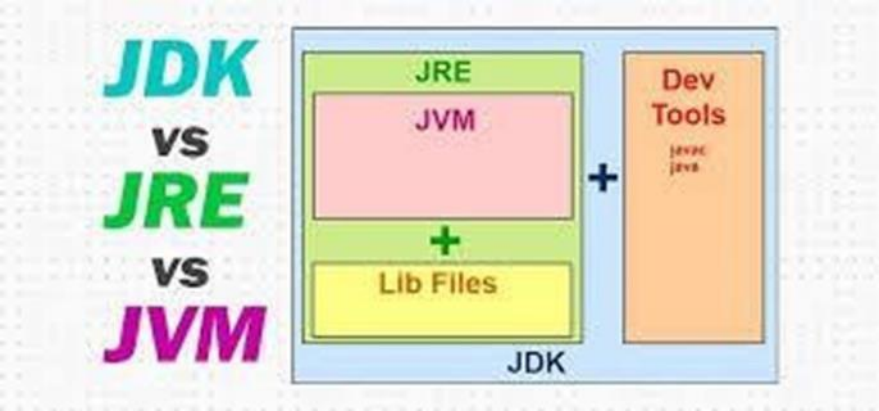
- Portable:

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

- Multithreaded:

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads.

JDK (Java development kit)



JDK = JRE(Java run time environment)+ Developmental tools JRE = JVM(Java virtual machine)+Library files

- Developmental tools includes javac, java etc. Javac acts as the compiler that converts high level language to machine language.
- Compiled file is sent to JVM.

Installation and configuration

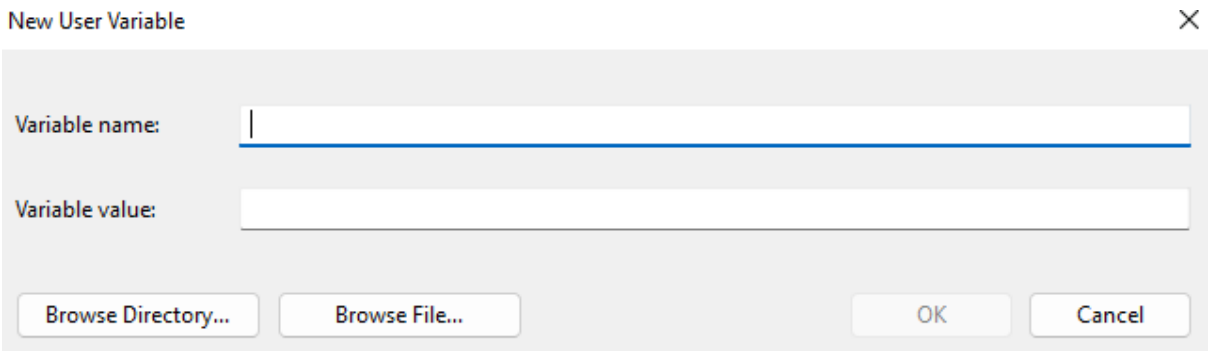
Windowsx64 is to be used for 64 bit PC.

Linux macOS Windows

Product/file description	File size	Download
ARM64 Compressed Archive	172.65 MB	https://download.oracle.com/java/17/latest/jdk-17_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	172.40 MB	https://download.oracle.com/java/17/latest/jdk-17_linux-aarch64_bin.rpm (sha256) (OL 8 GPG Key)
x64 Compressed Archive	174.03 MB	https://download.oracle.com/java/17/latest/jdk-17_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	149.43 MB	https://download.oracle.com/java/17/latest/jdk-17_linux-x64_bin.deb (sha256)
x64 RPM Package	173.77 MB	https://download.oracle.com/java/17/latest/jdk-17_linux-x64_bin.rpm (sha256) (OL 8 GPG Key)

Once installed the kit needs to be configured with the PC using the steps below.

Settings ⟶ Edit the system environment variables ⟶Environment variables New ⟶Variable name ⟶ Variable Value:C folder-program files -copy the address and paste the value.



Rules for naming a class:

1. Name must start with upper case letter.
2. The name must not contain any space & should not start with special characters
   Eg:AdditionOfTwo
3. Save the file by giving the class name as file name.
   It should be a noun

Sample program

```
class Sample{ → classname

public static void main(String arg[])
{
System.out.print("Hello java");

}
}
```

- Save the file as Sample.java, where Sample is the class name
- Always ensure that file name is same as the class name

while the file is saved.

- C:\Users\user>g:
- G:\ >cd javapg
- G:\javapg>javac Sample.java Compilation
- G:\javapg>java Sample Run
- G:\javapg>cd.. To go back to the folder

Here G is the drive with the folder whose folder name is javapg inside which the pgm Sample is saved.

Errors

2 types: **compilation time error & run time error**

- Compilation time error: deals with syntax mistakes
- Run time error:deals with logical mistakes eg: division by zero

Scanner

- We can read input from the console
- It is achieved by making use of an inbuilt class called scanner class.
- import java.util.Scanner; must be included outside the class

Syntax is given below

```
Scanner s=new Scanner(System.in);
int a=s.nextInt();
```

- Here instead of int different data types can also be used

float e=s.next Float();
double c=s.next Double();
string e=s.next
OR
string e=s.nextLine();

Data types

2 types: **Primitive & Non primitive**

- Primitive- int , long, short, byte, char, boolean, float, double
- Non primitive-String, Array ,Integer

Non primitive data types are also known as **wrapper classes or predefined classes**

Variable

3 types-local, instance, static

- **Local**- declared inside method
- **Instance**-declared outside method and inside class

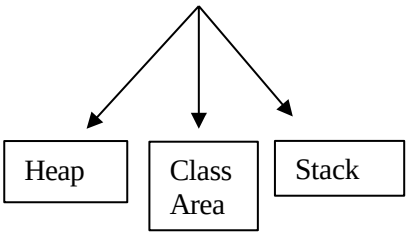
e 10

e 20

*The value 10 and 20 gets stored in two different locations in **heap or stack**

- **static**- declared with static key word. eg: static int f

*It saves memory
* It gets memory only once in **class area**(memory in JVM)



e | 20 |

The value 20 gets stored after erasing the value 10 that was previously stored in e .

- **println** ensures that the next output is printed on the next line.

Default values:

- It is **taken** by instance variable only
- Local variable **will not take** instance value
  int-0
  short-0
  byte-0
  long-0L
  float-0.0f
  double-0.0d
  String-null
  char-\u000
  boolean-false

Instance Variable	Static Variable
• Each object will have the copy of instance variable,it won't reflect on other objects • It is declared outside method and inside class • The value gets stored in different memory locations in heap or stack	• It saves memory • If any object changes the value of static variable it will retain the changed value • It is declared with the keyword "static" • The value gets stored in the same memory location of class area after every change in value .

Conditional Statements

1. Simple If

```
if(condition)
{
True
}
```

2. If Else

```
If(condition)
{
True
}
Else
{
False
}
```

3. If else if

```
if(condition1)
{

}
elseif(condition2)
{

}
else if(condition n)
{

}
else
{
```

```
}
```

4. Switch

```
Switch(expression)
{
case x:
//code block
break;
case y:
//code block
break;
default:
//code block
}
```

Loops

1. For Loop

```
for(declaration/initialisation;condition;increment/decrement)
{

}
```

2. While Loop

```
while(condition)
{

}
```

- It is entry controlled

3. DoWhile Loop

```
do
{

}
```

while(condition);

- It is exit controlled
- It runs the loop once without checking the condition due to the usage of “do” initially

Continue and Break Statements(Branching statements)

continue is used to skip the remaining statements in the loop and to continue with the next iteration.

- break statement stops the iteration once and for all and hence no more execution occurs.

Java Comments

1. **Single Line**- is turned into a comment using double slash(//) at the beginning

eg: int a=10; //variable declaration and initialization

2. **Multi Line**- is turned into a comment using /* at the beginning and */ at the end

```
eg :    /* int i;
        char a,b;*/
```

Array

- It stores elements of **similar** data type
- It is an object
- It is **index based**: First element of array is stored in 0 index

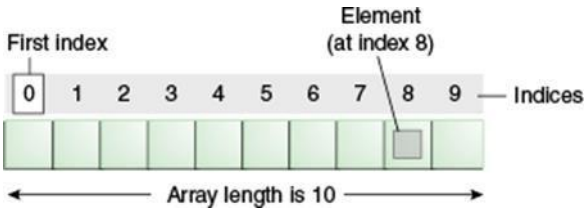
Advantages:

Code optimization: We can retrieve or sort data efficiently.
Random access: We can get any data located at an index position

Disadvantages:

Size: We can store only a fixed set of elements in a java array

Arrays are of **2 types**: **Single dimensional** array and **multi dimensional** array



Single Dimensional Array

Declaration:

Datatype[] a; ⟶ square bracket close to datatype. eg: int[] a;

OR

Datatype []a; ⟶ square bracket close to variable. eg: int []a;

OR

Datatype a[]; ⟶ square bracket after the variable. eg: int a[];

Instantiation:

Arrayref variable[] = new datatype[size]

Eg: int a[]=new int a[3];//declaration and instantiation

Intialisation:

a[0]=2;
a[1]=4;
a[2]=10;

Declaration,Instantation & Intialisation together

int a[]={2,4,10}

Multidimensional Array

Declaration:

Datatype[] [] array ref variable - eg: int[] [] a;
OR
Datatype [] []array ref variable - eg: int [] []a;
OR
Datatype array ref variable[][] - eg: int a[][];
OR
Datatype []array ref variable[] - eg: int []a[];

Instantiation:

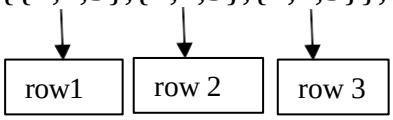
int [] []a = new int [3][3]a//declaration and instantiation

Intialisation:

a[0][0]=2;
a[0][1]=4;
a[0][2]=5;

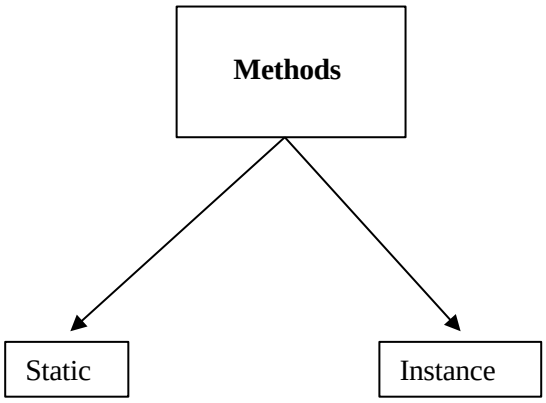
Declaration,Instantation,Intialisation together

int a[][]={{1,2,3},{2,4,5},{4,4,5}};



Methods

Methods- **2** types: **Static** & **Instance**
Static- invoked by using class
Instance/Non static -invoked by using objects



Syntax of method

Acess modifier nonacess modifier return type method name (parameter list)

{

//method body

}

eg:public static void add(int a)

{

}

Access Modifier:

*To set visibility
*eg:public,protected,default,private

Non Access Modifier:

*To set restriction
*Static,abstract,final
*If **non access modifier is not specified** method is **instance method**
Return type:

*returns value
*void,int ,char

<u>Rules for naming a method:</u>

1. Name must start with lower case letter
2. If it contains multiple words then first word will have lower case letter and remaining will start with upper case letters.
3. Eg:getDataVariable
4. The name must not contain any space & should not start with special characters
5. It should be a verb eg:main(),calculate()

<u>Parameterized static method/Method with an argument</u>

- Method takes an argument from the main and passes a value to the main
- The value passed to the main can also be used as an argument to another method
- The static method is invoked as **classname.methodname();**

Instance Method:

- **Creation of object** needs to follow the syntax given below:

ClassName Objectname = new ClassName();

Object declaration Instantiation Initalisation/Constructor invoking

Instantiation: It refers to the memory allocation for object,new reserves a memory .
Initialisation- Constructor stores a copy of argument in new.
int b - implies **variable** b declaration
Integer b –implies **Object** b declaration
Instance method is **invoked** as
Objectname.method Name();

Constructor

- It is used to **initialise** a object
- **2 types** of constructors

Default constructor-constructor with no arguments

Parameterized constructor- constructor with arguments

Syntax of default constructor :

```
Acess modifier   className()
{

}
```

Synatx of parameterized constructor:

```
Acess modifier   class name(arguments/parameter list)
{

}
```

Methods	Constructor
• Defines behaviour of object	• To initialize an object
• Return type present	• Return type absent(will not return value to main method)
• It can take any arbitrary name as method name	• It must use class name as method name always
Method must be created explicitly(we need to create it)	• default constructor will be provided by java compiler implicitly(compiler creates the constructor method)

Method Overloading

- If a class have multiple methods with same name but different parameters it is called **method overloading.**
- If a class have different methods with **same name and arguments of same number and data type** then it is impossible to apply method overloading.
- Method overloading is also **not possible** by **changing** the **return type** of the method alone.
- Method overloading takes place in **static and instance methods.**
- **Use of method overloading-**To increase readability of program.
- **Example** of method overloading

```
Class MethodOverloading
{
public static void add(int a,int b)
{    }
```

Method overloading cannot be achieved

```
        public static void add(int c,int d)
        {
        }
        public static void add(int c,int d)
        {
        }
    }
```

Techniques to perform method overloading

- Change the **number of arguments**.
- Change the **data type.**
- **Example**

```
Class Method Overloading
 {
public static void add(int a,int b)
{
int c=a+b;
System.out.println(c);
}
public static void add(int a,int b,int c)//different number of arguments
{
int c=a+b+c;
System.out.println(c);
}
public static void add (int a,float b)//different data type
{
 float c=a+b;
System.out.println(c);
}
}
```

Method Overloading is possible

Note

- String grade= "Grade is A" → **Sentence** cannot be returned return(grade)
- String grade= "Grade " → **word** can be returned return(grade)
- String grade= "A" → **character** can be returned return(grade)

Eclipse

Insatallation

www.eclipse.org → Eclipse IDE for Java Developers

↓

Eclipse photon(4.8)

↓

Eclipse IDE for Java Developers

↓

Windows 64 bit

↓

Download

Debugging

- Debugging can immediately report an error condition whenever it occurs. It prevents hampering the result by detecting the bugs in the earlier stage, making software development stress-free and smooth.
- **Step into** - Tells the program to execute, but only to the first line of the method call-(**f5 key**)
- **Step over** - Tells the program to execute but not stop in any methods. If I have a breakpoint on line 7 and tell the debugger to "step over", the debugger will then be on line 8. Choosing "step over" again will bring the debugger to line 9.-(**f6 key**)
- **Step return** - Tells the program to run to the end of the method and go back to the caller.(**f7 key**)
- **Resume** - Tells the program to keep going until it hits another breakpoint or completes.

This Keyword

- **this** is a reference variable that refers to the **current class object**.
- **To refer instance variable of current class:**by using the statement **this.instance variable** the instance variable can be used as a local variable.
- **To invoke current class method**:A particular method A can be

given a higher priority /invoked first before invoking another method B by using the statement **this.method name();** in another method B

- **To invoke current class constructors:** A particular constructor A can be given a higher priority /invoked first before invoking another constructor B by using the statement **this(argument of constructor);** in constructor B.

Note

- 2 constructors with the **same argument** is **not** possible ie constructor A and constructor B cannot have string argument
- **this** must be included as the first statement whenever it is used

String

Java String :Non Primitive Datatype

- A java string is same as array of characters

- char c[]={'j', 'a', 'v', 'a'}; String s=new String(c);

  OR

  String s=new String("java"); OR

  String s= "java";

Two ways to create string object

- By new keyword
- By string literal

By String Literal

String s= "java";
String s2= "java";

- Here an **object s2** is **not created** as it finds string **object s1** with value "java" hence s2 **acts as reference variable** and points to java. Thus it

doesn't create new instance but it creates a reference variable.

- Each time a string literal is created , JVM checks the string constant pool. If the string already exists in the pool ,a reference to the pooled instance is returned else a new string instance is created and placed in the pool.

- The usage of string literal thus makes java more **memory efficient**.

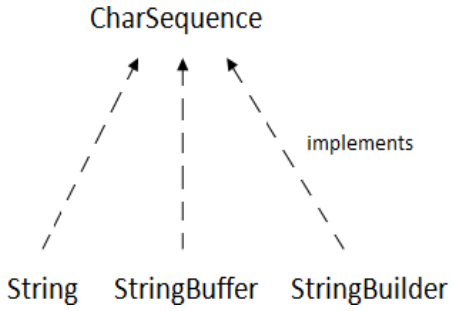
By new keyword

String a =new String("java");
String a1=new String("java");

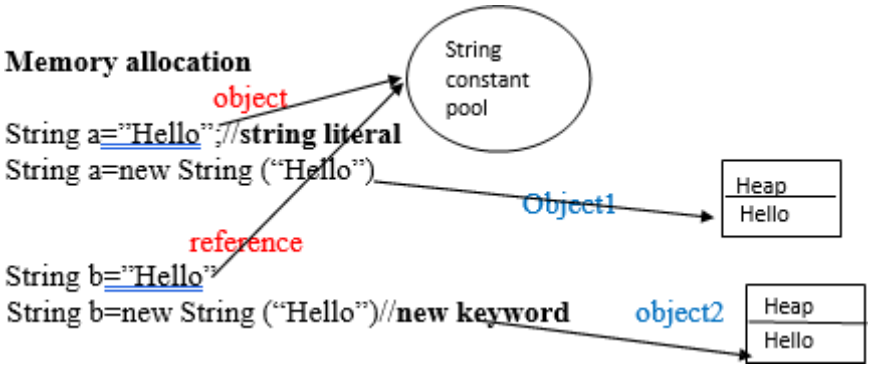
- It creates 2 objects and one reference variable.
- JVM creates a string object in the normal(non pool heap memory) and a literal "java" will be placed in the string constant pool.

String class, String buffer class and string builder class

- String is basically an **object** that represents **sequence of char values.**
- An array of characters works same as Java string



- We can create strings in java by using these three classes.
- The **CharSequence interface** is used to represent the sequence of characters
- String, StringBuffer and StringBuilder classes implement it.
- The **Java String is immutable** which means it cannot be changed



Concatenation of String

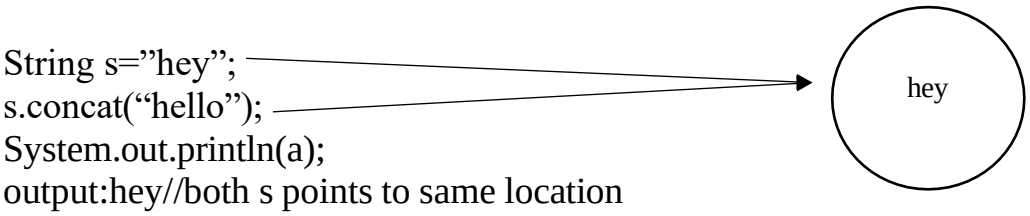
- 2 methods- By + (string concatenation) operator
  - By concat() method

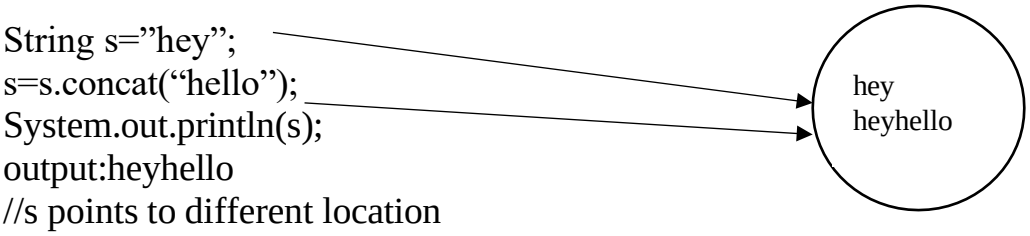
- String concatenation is implemented through the **StringBuilder (or StringBuffer) class**
- String concatenation operator produces a new string by appending the **second** operand onto **the end of the first** operand

- The string concatenation operator can concat **not only string but primitive values also**

String a=50+40+"hello"+20+20;
Output:90hello2020

Example of concat method:





String Buffer Class

- Java StringBuffer class is used to create mutable (modifiable) string.
- Java StringBuffer class is **thread-safe** i.e. multiple threads cannot access it simultaneously and **synchronised**

4 methods

- **append**-The append() method concatenates the given argument with this string.

- **insert**-The insert() method inserts the given string with this string at the given position.

- **delete**-The delete() method of StringBuffer class deletes the string from the specified beginIndex to endIndex.

- **replace**-The replace() method replaces the given string from the specified beginIndex and endIndex.

String buffers=”hello”;
s.append(“hey”);output=hellohey
s.replace(1,3,”java”);output=hjavalo
s.insert(1,”java”);output=hjavaello
s.delete(1,3)output=hlo

String Builder Class

- Java StringBuilder class is used to create **mutable (modifiable) string**.
- The Java StringBuilder class is same as StringBuffer class except that it is non synchronized and not thread safe.
- It includes the same **4 methods** given above:append(),delete(),insert(),replace()

Difference between String Buffer and String Builder

String buffer	String builder
• StringBuffer is **synchronized i.e. thread safe**. It means two threads can't call the methods of StringBuffer simultaneously.	• StringBuilder is **non-synchronized i.e. not thread safe.** It means two threads can call the methods of StringBuilder simultaneously.
• StringBuffer is **less efficient** than StringBuilder.	• StringBuilder is **more efficient** than StringBuffer.

Difference between String and String Builder

String	String buffer
• String class is immutable.	• StringBuffer class is mutable.
• String is slow and consumes more memory when you concat too many strings because every time it creates new instance.	• StringBuffer is fast and consumes less memory when you concat strings.
• String class overrides the equals () method of Object class. So, you can compare the contents of two strings by equals () method.	• StringBuffer class doesn't override the equals () method of Object class.

Access Modifiers

4 types- private, default, protected, public

- **Private**-accessible only within the class
- **Default**-accessible within the package
- **Protected**-accessible within the package and outside the package

through inheritance only

- **Public**-accessible everywhere

private public

Lowest Visibility Highest visibility

- **Child class method** must always **have higher visibility** than parent class method.

Main method usually comes along with child class

Inheritance

- It is the mechanism in which one class acquires the property of another class i.e a new classes are built based on the existing classes.

- In inheritance one object acquires all the properties and behaviour of the parent object.

- We can reuse **fields(variable)** and **methods** of existing class(parent class).

- Hence inheritance facilitates **code reusability**.

- Inheritance represents **IS-A** relationship(parent-child relationship)

- **Use of inheritance**-code reusability & method overriding

- 3 types- **Single** inheritance, **multilevel** inheritance, **hierarchal** inheritance

Syntax:

class **child class** extends **parent class**
{

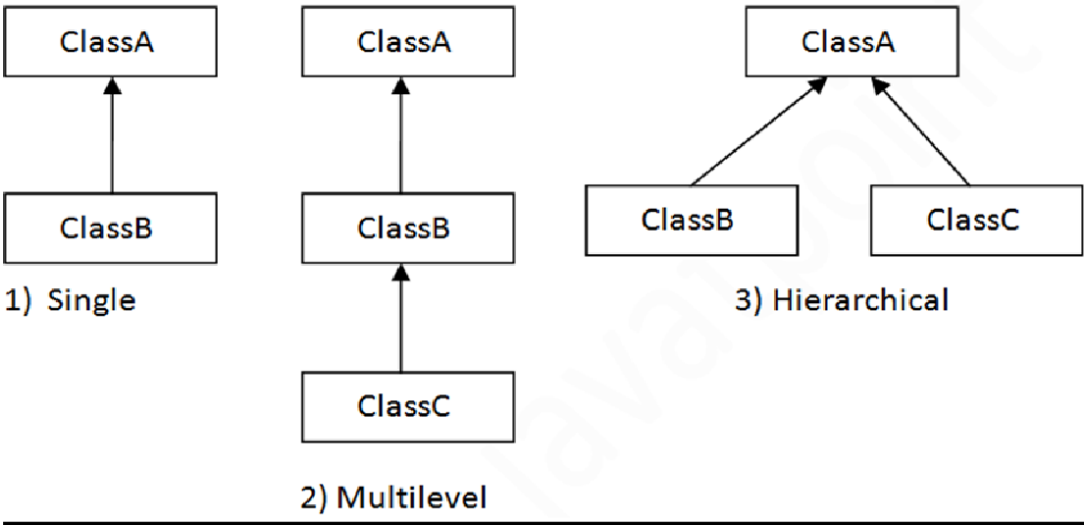
}

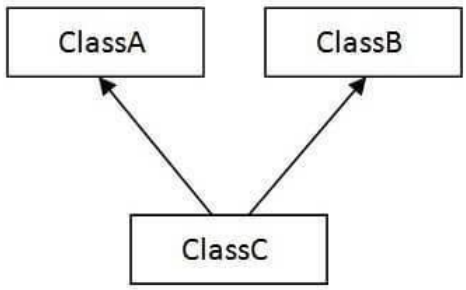
- Keyword "**extends**" is used by child class to inherit features of parent class.
- Child class is also known as **derived class, extended class or subclass**.
- Parent class is also known as **base class or super class.**

Types of Inheritance

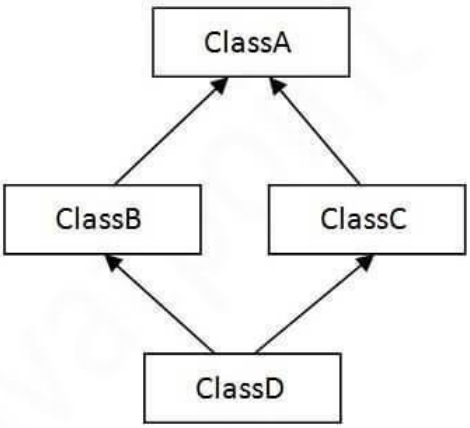
- Basically 3 types supported by java -single ,multilevel, hierarchal.

- Hybrid and multiple inheritance is **not supported** in java through **class** and is supported only through **interface.**





4) Multiple



5) Hybrid

.

Single Inheritance

```
class A
{
}
class B extends A
{
}
```

Multilevel Inheritance

```
class A
{
}
class B extends A
{
}
class C extends B
{
}
```

Hierarchical Inheritance

```
class A {
}
Class B extends A
{
```

```
}
Class C extends A
{
}
```

"Super" keyword

- **Super** is a reference variable used to refer immediate parent class object.
- **Super is used to refer immediate parent class instance variable:** when the same variable is used in the parent and child class as instance variable. The syntax **super. instance variable** print value of instance variable in parent class.

- **Super can be used to invoke parent class method:** The syntax **super. method name** will invoke the method of the parent class. When the child class has the same method as the parent class and the method is overridden, super keyword can be used to invoke parent class method.

- **Super is used to invoke the parent class constructor:** The syntax is **super (constructor argument).**

Note

- Super is added in each constructor automatically by the compiler if there is no super()or this()
- Super() must always be included as the first statement in method or constructor.

super	this
• Allows to access public, protected method ,data members, constructor of current class • Cannot access private field and methods	• Allows to access methods, datamembers ,constructors of current class including its own private method and data

Aggregation

- If a class have an **entity reference** of **another class** it is called aggregation.
- **Use of aggregation**-code reusability.
- It represents **HAS-A** relationship.
- Wherever a method needs to be used an object will be created for the specific method with reference to its class

Polymorphism

- Polymorphism is the concept in which a **single action** can be performed in **different ways**.
- **2 types of polymorphism**-compile time polymorphism and run time polymorphism.
- **Example of compile time polymorphism** -method overloading
- **Example of run time polymorphism** -method overriding

Method Overriding

It is the process in which a **child class** will have the **same method** as declared in the **parent class** i.e the **method name, number of arguments and data type** of the argument remains similar to the parent class.

Uses of method overriding

1. For the specific implementation of a method that has been already declared by its parent class.
2. For runtime polymorphism

Rules of method overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

@override annotation

@Override annotation is **used over methods** re-defined in a **chid class** which might be defined in the parent class. **child class** will have the **same**

method as declared in the **parent class** i.e the **method name, number of arguments and data type** of the argument remains similar to the parent class.

Uses of override annotation

1. **Improves code readability**. One can easily figure out that this method also exists in a super class.

2. **Ensures perfect overriding**: Applying @Override annotation forces the compiler to check if the **arguments** of the overridden method(number and their type) **match with that of super class** . Also, whether its **return type and access modifier matches** with that of **super class**.

If this annotation is missing and the method signature of the overridden method(in child class) does not match with super class method, then it will be considered as **a separate(or new) method which is defined in child class but is not present in parent class**.

3. **Prevents code break due to maintenance activities**. Suppose a method defined in a super class is overridden in a sub-class. Tomorrow, the method signature of super class changes to include one more argument. If the sub-class method has @Override annotation, you will get compiler error and hence code can be corrected. In absence of this annotation, again there will be no compile time error and the method in sub-class will be a separate method which may result in runtime errors.

Method overloading	Method overriding
• It is used to increase readability of the program	• It is used for the specific implementation of a method already provided by the super class
• It is performed **within** the **class** • Parameter must be different • method overloading can't be performed by changing return type of the method only. **Return type can be same or different in method overloading but parameter needs to be changed** • Example of **compile time polymorphism**	• It is performed in **two classes** that have (IS-A) **inheritance relation.** • Parameter must be **same** • **Return type** must be **same** • Example of **run time polymorphism**

Encapsulation

- It is the mechanism of **wrapping code**(methods) and **data**(variables) together as a single unit.
- A fully encapsulated class is created by the following steps
- declare **all its variable as private**
- use **setter** and **getter methods** to set(**modify**) and get(**view**)the data in it.

Advantages of Encapsulation

- By using a getter or setter method it can be made as **read only or write only class**
- **Data hiding**: Other class will not be able to access the data through private variables and hence it acts as protective shield.
- **Control over data**: Setter method gives control over the data as conditional statements can be used within it to select specific data values.

- **Loose coupling** can be achieved by encapsulation.

Final Keyword

- It is used to restrict the user from editing the data.
- It can be used along with **variable, method and class**
- **Final variable-value** will be constant and **cannot be changed**. A final variable which is not initialise is called a **blank final variable** and needs to be initialised in a constructor or static block in the case of static method
- **Final method**-If a method uses the keyword final before it then it **cannot be overrided**. If there are 2 methods void run() and final void run() it will result in compile time error.
- **Final class**-If a class uses the keyword final before it then it **cannot be extended** by inheritance.
- **A constructor cannot be final** as it is never inherited.

Example

- **public static final int p=2; ⟶ static final public final int p=2; ⟶ instance final**

Abstraction

- It is the process of **hiding the internal details** and showing only the functionality to the user.
- **2 ways to achieve abstraction**-Abstract class and interface

Abstract Class

- It provides **0-100% data hiding** based on the methods used within the class
- The methods of an abstract class can be **abstract or non abstract methods**. It provides **100% data hiding** if **all the methods are abstract**.
- Abstract class **cannot** be **instantiated**.
- Abstract methods can be defined **only in abstract class**.
- Abstract class will have **constructor** and **static method**.
- The **definition** of all the abstract method must be **provided** in the extended **class which is not abstract**.

- The abstract method will just include the **signature of the method** without its definition
- Abstract class can have **final methods**

Syntax :

```
abstract class classname
{
abstract returntype methodname()
{

}
```

Example :

```
public abstract class Test
{
public void hello();//non abstract method
public void abstract hey();//abstract method
Test()//constructor
{

}
}
Class ABC extends Test
{
public void hey()
}
public void hello()
{

}
public static void main(()
{

}
```

Interface

- It is a blueprint of class.
- In interface all **methods** will be **public &abstract** without method

body and **variables** will be **public static and final by default**.

- It does not include **instantiation** and **constructor.**
- Java 8 supports **default and static methods** & Java 9 supports **private methods** also in interface.

Uses of interface:

1. To achieve **100% data hiding** since all methods are abstract here.
2. It helps to achieve **multiple inheritance.**
3. It helps to achieve **loose coupling**.

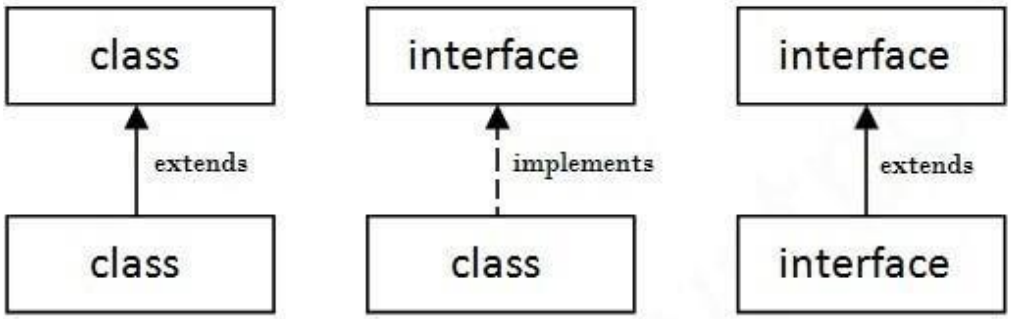
Note:

Loose coupling: It means reducing dependencies of a class that use different class directly .

Syntax:

```
interface interface name
{
//abstract methods
//public static final variable
}
```

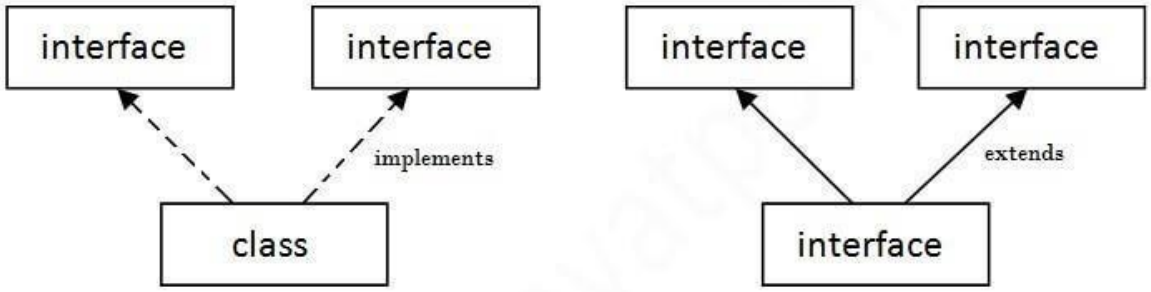
Relation between class and Interface



Example:

```
Interface Test
{
public static final a=10;
public abstract hello();
}
Class Abc implements Test
{
public void hello()
{
System.out.println("Hello");
}
public static void main(String args[])
{
Test ob=new Abc();
ob.hello();
}
}
```

<u>Multiple Inheritance</u>



Multiple Inheritance in Java

Example:

```
interface Printable
{
void print();
}
interface Showable
{
```

```
void show();
}
class A7 implements
Printable,Showable{ public
void print()
{System.out.println("Hello");
}
public void show()
{System.out.println("Welcome");
}

public static void
main(String args[]){
A7 obj = new A7();
obj.print();
obj.show();
 }
```

Difference Between Abstract and Interface

abstract	interface
• It can have abstract and non abstract methods.	• It can have only abstract methods.
• It doesn't support multiple inheritance.	• It supports multiple inheritance.
• It can have static,non static,final and non final varaiable.	• It can have only static and final variable.
• Abstract class can provide implementation of interface. Eg :abstract class Classname implements interface	• Interface cannot provide implementation of abstract class Eg: Interface interfacename implements classname (not possible
• Abstract keyword is used to declare abstract class	• Interface keyword is used to declare interface.
• An abstract class can extend another Java class and implement multiple Java interfaces.	• An interface can extend another Java interface only

Difference Between Abstract and Final

Abstract	final
• It can be inherited to subclasses • It can be altered in subclass • It cannot be instantiated • It can contain abstract methods • Extra functionality to the methods can be added • All methods need not be implemented	• cannot be inherited • final method cant be overridden • It can be instantiated • It must not include abstract methods • No extra functionality can be added • All methods should be implemented

Exception Handling

- It is an **unwanted condition** that disrupts the normal flow of program.
- It can be **handled**.
- It is a **class**
- 2 types of exception-**Checked and Unchecked**
- **Checked/compile time exception** -program doesn't run due to **external error** . It can be found and corrected at **compile time**

  Eg: Input-output exception-File cannot be read as path is different SQL exception-error in data base

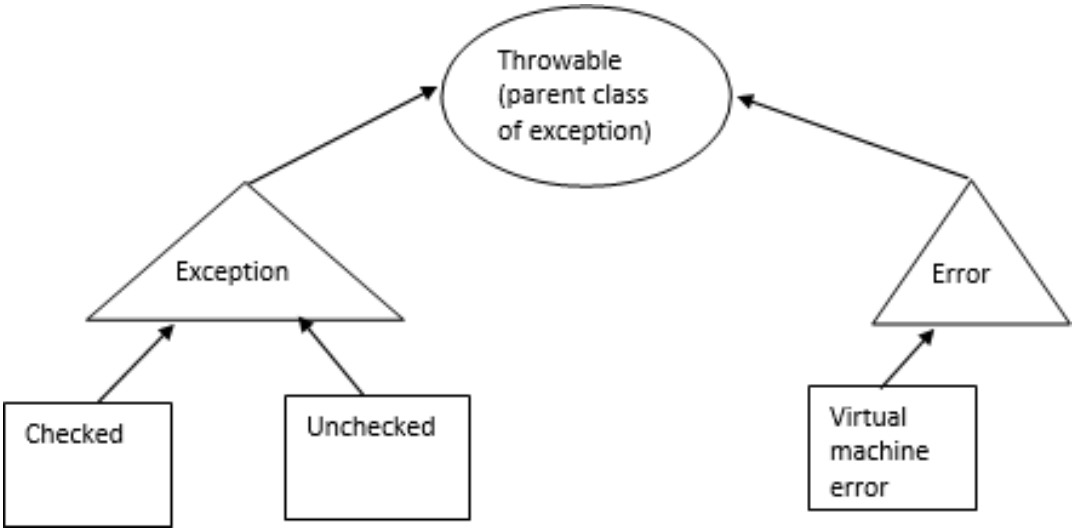
Unchecked/runtime exception-program doesn't run due to **error created by the programmer**. It cannot be corrected in compile time as issues occur in run time.It is corrected at **run time**

Eg: Arithmetic exception

Array out of bound

- Compile time exception is **mandatory** and runtime exception is **optional**
- Class Exception is the **parent class** and all other classes under it are taken as child class
- Eg:Class SQL exception extends Exception

ClassFileNotFoundException extends Exception



Java Try Catch Block

- Java **try** block is used to enclose the code that might throw an exception
- It must be **used within the method**
- If an exception occurs at the particular statement of try block, **the rest of the block code will not execute**. So, it is better not to keep the code in try block that will not throw an exception.
- **Java try block** must be followed by either **catch or finally block.**
- **Java catch block** is used to handle the Exception by declaring the type of exception within the parameter
- The catch block must be used **after the try block** only
- A try statement can have **multiple catch blocks.**
- A **try catch** block comes only within **psvm.**
- JVM has a default try catch block which handles exception in case the exception is not handled

Syntax

```
try
{
//code that may throw an exception
}
catch(Exception_class_Name    ref)
{

}
```

OR

```
try
{
//code that may throw an exception
}
finally
{
}
```

Example

```
public class TryCatchExample2 {

public static void main(String[] args) {

try
{
int data=50/0; //may throw exception
System.out.println("exception")
}
catch(ArithmeticException e)
{
System.out.println(e);
}
System.out.println("rest of the code")
}
}
```

<u>Multiple Catch Block</u>

- At a time only one exception occurs and at a time only one catch block is executed.

- All catch blocks must be ordered from **most specific to most general**, i.e. catch for ArithmeticException must come before catch for Exception,else it results in compile time error

- In case there are **two exceptions** within the **try** block, the exception which is listed first in the catch block alone will be

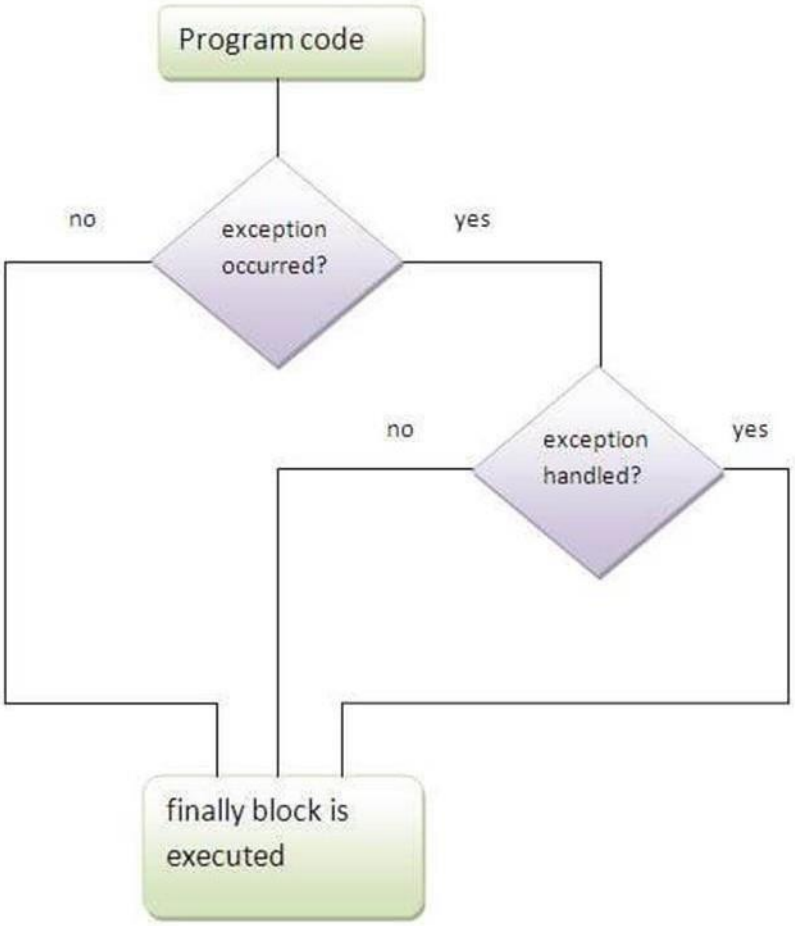
handled.

- In case there are two exceptions and **both are not listed in catch block** then the catch block with the general class exception gets invoked

Java Nested try block

- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. Here we make use of nested try block.

Java Finally Block



- Java finally block is a block that is used to execute important code such as closing connection, stream etc
- Finally ensures **mandatory execution** of code within it.
- Java finally block is **always executed whether exception is**

handled or not.

- Java finally block **follows try or catch block**
- If you don't handle exception, before terminating the program**, JVM executes finally block(if any).**
- **Use of finally**-to close a file or connection

Note

- For each try block there can be zero or more catch blocks, but only one finally block.
- The finally block will not be executed if program exits(either by calling System.exit() or by causing a fatal error that causes the process to abort).

Java Throw Keyword

- throw keyword is used to throw an exception
- It can be used to throw checked or unchecked exception

Syntax
throw new exception;

- It is used mainly for **custom exceptions**(exceptions created by us)

Example

```
class Mickeymouse exception extends Exception
{
void validate(int age){
if(age<18)
throw new Mickeymouse Exception("not valid"); else
System.out.println("welcome to vote");
}
public static void main(String args[]){
Mickeymouse ob=new Mickeymouse();
ob.validate(13);
System.out.println("rest of the code...");
}
}
```

Output
Exception in thread main java.lang.MickeymouseException:not valid

Java Throws Keyword

- It is used to **declare an exception**
- It gives an information to the programmer that there may occur an exception
- It is mainly used to handle checked exception as unchecked exception handling is programmers responsibility

Syntax

```
returntype methodname() throws exceptionclassname
{
//method code
}
```

- Exception is thrown from method to main and main to JVm
- When it is thrown from method, try catch block within psvm should handle it
- When thrown from main ,JVM handles it by using the default try catch block

Example 1

```
Class Test{
Public void hello() throws IO Exception file
{

}
public void print()
{

}
public static void main(String args[])
 {
 Test ob =new Test();
Ob.print();
  try
{
Ob.hello();
}
catch (Exception e)
{

}
```

Throw	Throws
• Java throw keyword is used to explicitly throw an exception • Checked exception cannot be propagated using throw only. • Throw is followed by an instance. • Throw is used within the method	• Java throws keyword is used to declare an exception. • Checked exception can be propagated with throws. • Throws is followed by class. • Throws is used with the method signature.

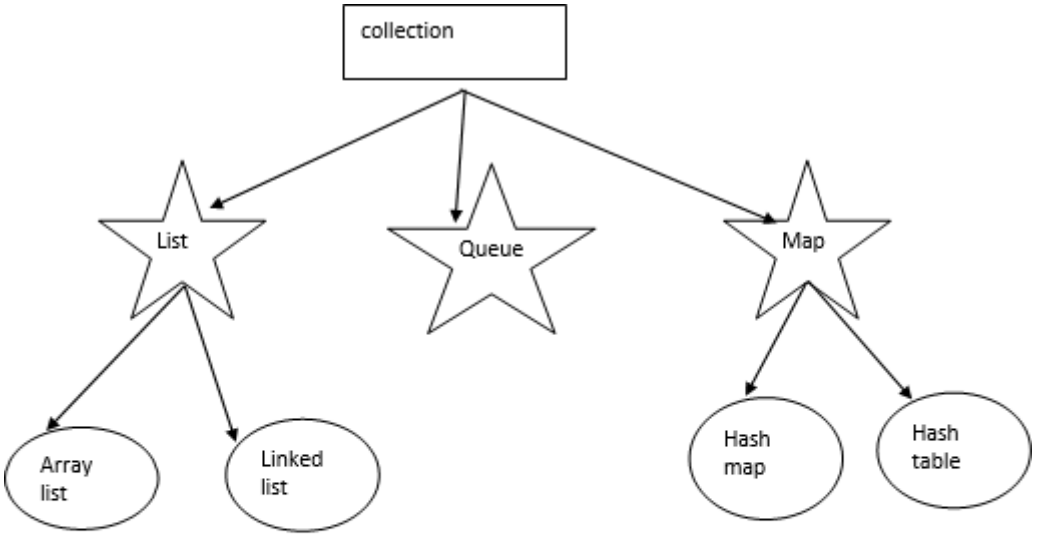
Difference Between Final, Finally and Finalized

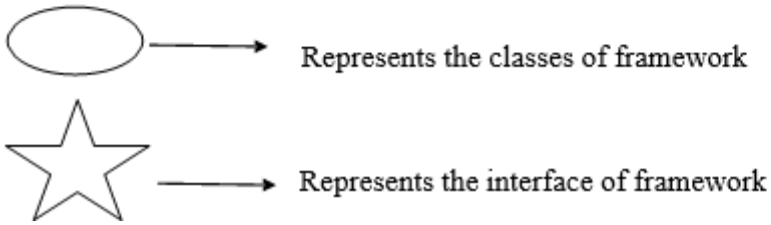
final	finally,	finalize
• Final is used to apply restrictions on class, method and variable. • Final class can't be **inherited** • final method can't be **overridden** • final variable value **can't be changed** • Final is a **keyword.**	• Finally, is used to place important code, it will be executed whether exception is handled or not. • Finally, is **a block**	• Finalize is used to perform clean up processing just before object is garbage collected. • Finalize is a **method.**

Collections

- Collection in Java is a **framework** that provides an architecture to store and manipulate the **group of objects**.

It includes **interfaces**(list, queue, map) and **classes**(array list linked list etc)





Note:

jar-java archive file

List Interface

It includes list type data structure in which we can **store** the ordered collection of objects. It **can have duplicate values**.

Syntax

1.List <data-type> a1= **new** ArrayList();
2.List <data-type> a2 = **new** LinkedList()

- It is mainly used to insert, delete, and access the elements from the list

ArrayList

It uses a dynamic array to store the element of different data types
It maintains the insertion order based on array index.

Syntax

ArrayList<String> a1=**new** ArrayList<String

OR

List<String> a1=**new** ArrayList<String>();(>();(Generics/data type ⟶ JDK 1.5 feature)

OR

List<String> a1=**new** ArrayList();(generic on RHS absent-JDK 1.7feature)

- a1.add(20) ⟶ places the value 20 in the next index position of array
- a1.remove(2) ⟶removes the value stored in 2nd index position of array
- a1.size() ⟶ returns size of array
- a2.addall(a1)⟶ copies all values of a1 to a2

Advanced for loop

```
for(Integer i:a1)
{
system.out.println(a1.add(i))
}
//prints the values added to array
```

- If a1.add(20) representing integer datatype and a1.add("abc") representing string data type needs to be printed with the same for loop then the following syntax is used :
- for(object i:a1)
  {system.out.println(a1.add(i))
  }//prints the values added to array

LinkedList

- It performs the same operations as an array list
- However **linked list is less efficient than array list** due to thread issues

Syntax

```
LinkedList<String> al=new LinkedList<String>();
                    OR
List<String> al=new LinkedList<String>();
                    OR
LinkedList<String> al=new LinkedList();
```

Iterator interface

- Interface with methods hasNext(),next(),remove()
- Iterator permits the caller to remove the given elements from the specified collection during the iteration of the elements.
- **hasNext()-**Returns a true value if the more number of elements are encountered during iteration.
- **next()-**Returns the next specified element during the iteration.
- **remove()-**Removes the last element from the collection as provided by the iterator.

Example:

```
List<String> list = new LinkedList<>();
list.add("Welcome");
list.add("to");
list.add("our");
list.add("website");
Iterator<String> itr = list.iterator();
```

↓

Class arraylist has a method iterate() which returns an object iterator itr referring to the interface iterator

```
//Returns true if there are more number of elements
while(itr.hasNext()) {

//Returns the next element.

System.out.println(itr.next());


}//Removes the last element.
itr.remove();
```

Apache POI

- Apache POI is an open-source java library to create and manipulate various file formats based on Microsoft Office. Using POI, one should be able to perform create, modify and display/read operations on the following file formats. For Example, Java doesn't provide built-in support for working with excel files, so we need to look for open-source APIs for the job.
- Apache POI is able to handle both XLS and XLSX formats of spreadsheets.
- Apache POI provides excellent support for additional excel features such as working with Formulas, creating cell styles by filling colors and borders, fonts, headers and footers, data validations, images, hyperlinks, etc.

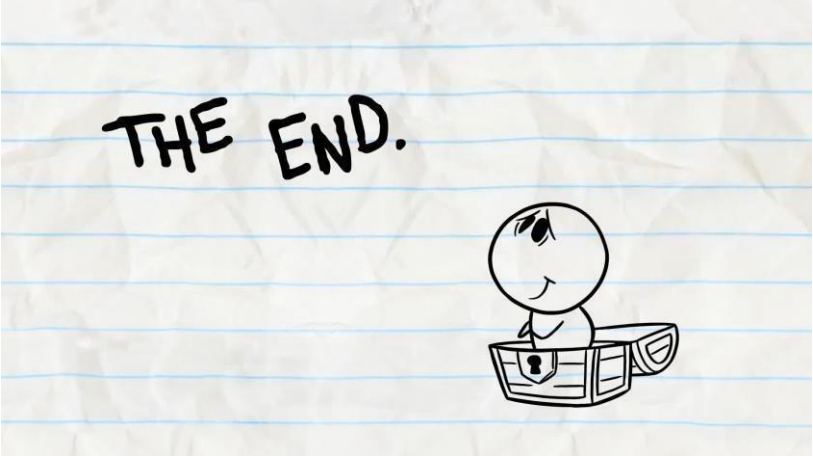
Jenkins

- Jenkins is an open-source automation server that facilitates the automation of several stages of the software development lifecycle, including application development, testing, and deployment.
- jenkins is flexible.
- You can add the n no.of plugins you want to add to the jenkins.
- You can automate the proceses of CI/CD pipelines of all the projects.
- Jenkins CI/CD stands for Continuous Integration / Continuous Deployment

WORKSHEET

1. Sample Hello world.
2. Write a program to add two numbers. Accept numbers using command line arguments.
3. Write a program to
   a) add two integer numbers
   b) add one float number and one integer number
   c) print your name and age
   d) calculate area of Triangle
4. Write a program to swap two numbers with temporary variable. Print the numbers before and after swap.
5. Write a program to swap two numbers „without" temporary variable. Print the numbers before and after swap.
6. Write a program to check the given number is positive.

7. Write a program to check whether the candidate is eligible for driving license.
8. Write a program to check whether the given number is Odd/Even.
9. Write a program to find largest of three Numbers
10. Write a program to find the grade of a Student based on total marks
    - Mark less than 40- Failed
    - 40 to 60–Grade D
    - 61 to 70-Grade C
    - 71 to 80-Grade B
    - 81 to 100-Grade A
11. Write a program to check whether the given character is Vowel/not (Use switch-case).
12. Write a program to print Fibonacci series[0 1 1 2 3 5].
13. Write a program to find the reverse of number.
14. Write a program to check whether the given number is Palindrome/Not.
15. Write a program to check whether the given number is Armstrong/Not.
16. Write a program to check whether the given number is Prime/Not.
17. Write a program to find the count of even numbers and odd numbers from 10 to 20.
18. Write a program to sum up all the elements of an array.
19. Write a program to add two matrices.
20. Write a program to search array element with Linear Search.
21. Write a program to sort array element with Bubble Sort.
22. Write a program to set your name and age by a static method and get them in an another static method.
23. Write a program to find the factorial of a number(**static method**)
    - Output – "Factorial of <given number> is <result>.
24. Write a program to check whether the given number is Palindrome/Not by using **static methods**.
    - Method 1- to find the reverse(pass the number in argument)
    - Method 2-to check palindrome/not
25. Write a program to check whether the candidate is eligible for Voting(Use **static method** and boolean return type).
26. Write a program to deposit and withdraw amount from bank account(Withdrawal amount do not exceeds the current balance). Add an extra method to check the account balance. (**static methods**)
27. Write a program to check whether the customer have discount (get 20% discount if total amount is greater than 5000) or not and get the final amount in main method. (**static methods**)
    - Get prices of items using parameterized method

- Method 1 - Calculate total amount
- Method 2 - Check discount

28. Write a program to find the
    a) Average of three integer numbers, three float numbers(should have same method name)
    b) Area of figures(circle, rectangle, square) by using three methods(should have same method name)
29. Write the above program with parameterized constructor (to calculate total amount). (**instance methods**)
30. Write a program to find the grade of 2 students based on total marks(3 subjects)
    - Get the students marks by constructor
    - Return total mark to in main method
    - Find the grade of each student.